Software Design Description (SDD)

Internship Application

1. Introduction

Purpose.

This document describes the software design of **Internship Application**. It provides the technical structure required to implement, test, and maintain the application.

Scope.

Internship Application is a web application that allows students to search and apply for internship offers, companies to publish offers and manage applications, and school staff to monitor students' internship search progress.

Audience.

Developers, testers, project members, and instructors.

References.

- Business Requirements Document (BRD) – Internship Application
- Software Requirements Specification (SRS) – Internship Application
- GDPR requirements
- Software Engineering course materials

2. System Overview

Internship Application is a web platform connecting students, companies, school staff, and administrators.

The application is divided into a frontend, a backend, and a database.

- **Frontend:** Provides the user interface.
- **Backend:** Handles application logic, authentication, offers, and applications.

- **Database:** Stores users, internship offers, applications, CV information, and notifications.

This corresponds to the role of the *System Overview* in the example SDD, which gives a concise description of the system before describing its architecture.

3. Architectural Views

3.1 Context View

The system interacts with four main types of users:

- **Students ↔ Internship Application:** Manage profiles, search offers, submit applications, and track their status.
- **Companies ↔ Internship Application:** Manage company profiles, publish internship offers, and review applications.
- **School Staff ↔ Internship Application:** Monitor students' internship search progress.
- **Administrators ↔ Internship Application:** Manage users, roles, and platform information.

3.2 Component View

The main logical components are:

- **AuthService:** Handles registration, login, authentication, and user roles.
- **StudentService:** Manages student profiles, skills, and CV information.
- **CompanyService:** Manages company and recruiter information.
- **OfferService:** Handles internship offer creation, modification, deletion, and search.
- **ApplicationService:** Handles internship applications and application status updates.
- **NotificationService:** Manages notifications related to applications.
- **MonitoringService:** Provides school staff with information about students' progress.
- **Database:** Stores all core application data.

Key Dependencies

- All services communicate with the database when necessary.
- `ApplicationService` uses information from `StudentService` and `OfferService`.
- `ApplicationService` triggers `NotificationService` when an application is submitted or its status changes.
- `MonitoringService` retrieves information about students and their applications.

Cette organisation reprend le principe du modèle, qui découpe le logiciel en services ayant chacun une responsabilité claire.

3.3 Deployment View

The application will use a simple three-layer architecture:

- **Frontend:** React web application.
- **Backend:** Node.js with Express.
- **Database:** PostgreSQL relational database.
- **Communication:** REST API using HTTPS and JSON.

The first version will be a web application accessible through a standard web browser.

4. Data Design

Core Entities

- **Student** (`id, first_name, last_name, email, password, field_of_study, skills, ...`)
- **Company** (`id, name, email, description, website, ...`)
- **InternshipOffer** (`id, company_id, title, description, location, duration, required_skills, status, ...`)
- **Application** (`id, student_id, offer_id, application_date, status, ...`)
- **CV** (`id, student_id, file_name, upload_date, ...`)

- **Notification** (`id, user_id, message, type, is_read, created_at`)
- **SchoolStaff** (`id, first_name, last_name, email, role, ...`)

Integrity Rules

- An internship offer must belong to one company.
- An application must reference one student and one internship offer.
- A student cannot apply more than once to the same internship offer.
- Each application must have a valid application status.
- Only authorized users can access personal data and CVs.

This follows the SDD example, which describes both the core entities and the main integrity rules of the database.

5. Interface Design

External REST Endpoints – Examples

Authentication

- `POST /api/auth/register` — Create an account.
- `POST /api/auth/login` — Authenticate a user.

Students

- `GET /api/students/{id}` — View a student profile.
- `PUT /api/students/{id}` — Update a student profile.

Internship Offers

- `GET /api/offers` — View available internship offers.
- `GET /api/offers/{id}` — View one internship offer.
- `POST /api/offers` — Create an internship offer.
- `PUT /api/offers/{id}` — Update an internship offer.
- `DELETE /api/offers/{id}` — Delete an internship offer.

Applications

- `POST /api/applications` — Submit an internship application.
- `GET /api/applications/{id}` — View an application.
- `PUT /api/applications/{id}/status` — Update the application status.

UI Surfaces

- **Student Interface:** Profile, internship search, applications, notifications.
- **Recruiter Interface:** Company profile, internship offers, received applications.
- **School Staff Interface:** Student monitoring and internship progress.
- **Administrator Interface:** User and role management.

Le modèle SDD utilise lui aussi des exemples d'endpoints REST suivis d'une description des interfaces principales.

6. Detailed Design

6.1 Component Responsibilities

ApplicationService

- Checks that the student and internship offer exist.
- Checks that the student has not already applied to the same offer.
- Creates and stores the application.
- Allows authorized recruiters to update its status.
- Triggers notifications when necessary.

OfferService

- Allows companies to create internship offers.
- Allows recruiters to modify or delete their offers.
- Provides internship offers to students.
- Handles search and filtering.

NotificationService

- Creates notifications when important events occur.
- Notifies a company when a student applies.
- Notifies a student when an application status changes.

MonitoringService

- Retrieves students and their application information.
- Allows school staff to monitor internship search progress.

6.2 Sequence – "Apply for an Internship"

1. The student selects an internship offer.
2. The frontend sends an application request to the backend.
3. `AuthService` verifies that the student is authenticated.
4. `ApplicationService` checks that the student has not already applied.
5. The application is stored in the database.
6. `NotificationService` creates a notification for the company.
7. The system confirms the application to the student.

6.3 Error Handling & Recovery

- Invalid user inputs return an appropriate error message.
- Unauthorized access attempts return an authentication or authorization error.
- Duplicate applications are rejected.
- If an internship offer does not exist or is closed, the application cannot be submitted.
- Unexpected server errors are logged for debugging.

The example SDD also describes selected component responsibilities, one important sequence, and basic error handling.

7. Security & Compliance

- User passwords must be securely hashed.
- Authentication will use secure user sessions or tokens.
- Role-Based Access Control (RBAC) will restrict features according to user roles.
- HTTPS will be used for communications.
- User inputs must be validated.
- Personal information and CVs must only be accessible to authorized users.
- Personal data must be managed according to GDPR requirements.

8. Performance, Scalability & Availability

- The application should respond within a few seconds under normal conditions.
- The backend should support multiple simultaneous users.
- Database queries should be optimized for commonly accessed data such as offers and applications.
- The architecture should allow new features to be added without major changes to existing components.

Pour votre projet, je ne mettrais pas de chiffres très techniques comme `p95 < 300 ms`, autoscaling ou read replicas, contrairement à l'exemple. Ceux-ci seraient difficiles à justifier à ce stade du TP. Le modèle les présente pour un système hospitalier beaucoup plus conséquent.

9. Observability

- The backend should log important operations and errors.
- Authentication errors and server errors should be recorded.
- Logs can be used by developers to identify and correct problems.

10. Internationalization & Accessibility

- The interface should remain simple and readable.

- Main actions should be accessible using clear buttons and labels.
- The application should be responsive and usable on different screen sizes.
- Additional languages may be added in future versions.

11. Design Rationale

- **React** is used for the frontend to build an interactive web interface.
- **Node.js and Express** are used for backend development and REST APIs.
- **PostgreSQL** is used because the application contains structured data with clear relationships between students, companies, offers, and applications.
- A modular architecture separates the main responsibilities and facilitates maintenance and future development.

Le choix React + Node.js est également cohérent avec la suite annoncée dans l'énoncé du TP, qui prévoit le développement pratique avec **Node.js pour le backend et React pour le frontend**.

12. Assumptions & Constraints

- The first version of Internship Application will be web-only.
- Users must have Internet access to use the application.
- The project is developed by a small student team.
- Development time is limited.
- Advanced features are not included in the first version.
- External recruitment platforms will not be integrated in the first version.

13. Open Issues / Future Work

Possible future improvements include:

- Matching students with internship offers based on their skills.
- Integration with external internship or job platforms.
- Email notifications.
- Direct messaging between students and recruiters.

- Internship agreement management.
- Advanced statistics for school staff.